



VNW Project

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

14 de junio de 2026

1. Equipo	1
2. Repositorio	2
4. Modelo computacional	3
4.1 Dominio	3
4.2 Lenguaje	3
5. Implementación	4
5.1 Frontend	4
5.2 Backend	5
5.3 Dificultades encontradas	6
6. Futuras extensiones	6
7. Conclusiones	6
8. Referencias	7
9. Bibliografía	7

1. Equipo

Nombre	Apellido	Legajo	E-mail
Mateo	Andrioli	64042	mandrioli@itba.edu.ar
Tomás	Kaneko	62297	tkaneko@itba.edu.ar
Antonio	Valentinuzzi	64751	avalentinuzzi@itba.edu.ar
Francisco	Peña	62431	fpenacritto@itba.edu.ar

2. Repositorio

La solución será versionada en: https://github.com/matsdah/TPE_TLA

3. Introducción

El trabajo desarrollado consistió en la creación de un motor gráfico orientado a la categoría de novelas visuales (*Visual Novels*) junto con su subsecuente lenguaje de programación.

Se buscó la creación de un compilador, el cual compila el lenguaje creado y genere un `play.sh` que se encargue de iniciar el motor y mostrar en pantalla la historia deseada.

El proyecto fue llevado a cabo en tres etapas. La primera con la propuesta de la idea y su planificación; la segunda, en la cual se desarrolló el analizador léxico y sintáctico, partiendo desde el proyecto base, dado por la cátedra; y la tercera, en la cual se transformó el lenguaje producido en una salida en C que permitiera *buildear* y ejecutar los archivos `.story`.

4. Modelo computacional

4.1 Dominio

El dominio seleccionado para el diseño e implementación de este Lenguaje Específico de Dominio (DSL) comprende la creación, estructuración y orquestación de Novelas Visuales (o *Visual Novels*) y videojuegos con una fuerte relación narrativa.

Una Novela Visual es un género de videojuego interactivo caracterizado por enfocarse casi exclusivamente en contar una historia a través de texto, acompañado de imágenes estáticas o animadas (fondos y *sprites* de personajes), efectos de sonido y música. La interactividad radica principalmente en la toma de decisiones por parte del jugador, lo que genera ramificaciones en la trama o *branching narrative*, afectando variables de estado internas (como la afinidad con un personaje, el karma, el progreso en una ruta específica, economía, etc.) y conduciendo a múltiples finales posibles.

El lenguaje propuesto abstrae las complejidades subyacentes del desarrollo de videojuegos (ciclo principal de juego o *game loop*, renderizado gráfico, gestión de recursos multimedia, manejo de eventos, etc.) y expone un modelo computacional centrado puramente en la narrativa. El lenguaje es de tipado estático con variables del tipo *strong typing*.

El dominio es abstraído a través de entidades de primera clase como: actores, escenas, recursos (o *assets*), flujos de decisión, o estados (variables numéricas que se modifican en base a las decisiones del jugador).

4.2 Lenguaje

El lenguaje está centrado en la creación de escenas y las acciones que estas pueden ejecutar, partiendo desde la escena *Start* y cambiando entre ellas. Además, se armó el lenguaje pensando en el manejo de los siguientes 3 recursos:

- *Characters*
- *Assets*
- *Ints*

A partir de estos tres recursos -su manejo, una implementación *if-else* y una función *goto* que permita el salto entre escenas- se desarrolló el lenguaje.

A continuación se deja un ejemplo que permite identificar las declaraciones y manejo de los recursos mencionados:

```
character "Narrator" as narr color #AAAAAA;
```

```
character "Aria"      as aria color #FF5733;
character "Guardian" as guard color #33AAFF;

asset bg_title = "../src/test/assets/img/background.png";
asset spr_aria = "../src/test/assets/img/character.png";
asset bgm_main = "../src/test/assets/audio/music.wav";
asset sfx_win  = "../src/test/assets/audio/effect.wav";

set score = 0;
set bonus = 2;

scene "start" {
  show background bg_title;
  play music bgm_main;
  narr "Welcome to the technical showcase.";
  show sprite spr_aria;
  aria "Every DSL feature will be exercised in sequence.";
  narr "Make a choice to mutate state variables.";
  choice {
    "Add bonus" {
      set score += bonus;
      aria "Score increased by bonus.";
      goto "Check";
    }
    "Subtract penalty" {
      set score -= 5;
      aria "Score decreased by penalty.";
      goto "Check";
    }
    "Double and add" {
      set score = (bonus * 3) + 1;
      aria "Score set via arithmetic expression.";
      goto "Check";
    }
  }
}
```

5. Implementación

Se usó como punto de partida el proyecto dado por la cátedra (8.1). A partir de este, se creó una copia en el repositorio del equipo.

5.1 Frontend

El proyecto inicial proporcionaba un analizador léxico, *Flex*, y un analizador sintáctico, *Bison*. Con ellos, se se desarrolló una gramática que satisfaga el lenguaje deseado y se determinó la necesidad de los siguientes símbolos:

Terminales: INTEGER, STRING, COLOR_LITERAL, IDENTIFIER, CHARACTER, AS, COLOR, ASSET, SCENE, SET, SHOW, HIDE, BACKGROUND, SPRITE, PLAY, STOP, MUSIC, SOUND, CHOICE, IF, ELSE, GOTO, END, ASSIGN, ASSIGN_ADD, ASSIGN_SUB, LESS, GREATER, LESS_EQ, GREATER_EQ, EQUAL, NOT_EQUAL, ADD, SUB, MUL, DIV, OPEN_BRACE,

CLOSE_BRACE, OPEN_PARENTHESIS, CLOSE_PARENTHESIS, SEMICOLON, OPEN_COMMENT, CLOSE_COMMENT, IGNORED, UNKNOWN.

No Terminales: program, declarationList, declaration, statementList, statement, dialogueStatement, showStatement, hideStatement, playStatement, stopStatement, gotoStatement, setStatement, choiceStatement, ifStatement, endStatement, optionList, option, condition, expression.

A partir de estos conjuntos de símbolos, se validaron las estructuras que complían con los requisitos del lenguaje y cumplieran con la gramática establecida. Sin embargo, para los casos de validación de archivos, no fue suficiente con solo esta implementación. Fue necesario la espera a la implementación del *backend* para la lograr verificar que las direcciones llevarán a archivos existentes.

5.2 Backend

Por la misma naturaleza del proyecto, se le tuvo que otorgar al lenguaje un entorno de *runtime*. Tanto el motor como la traducción del código, son ejecutados en C con ayuda de la librería de *Raylib*.

Debido a la necesidad de guardar diferente información, se implementó una tabla de símbolos con los siguientes tipos: *scene*, *character*, *asset*, *value*. Debido a que no consideramos a las escenas como funciones, el scope de las variables es global, cualquier *scene* puede acceder a cualquier variable. Sin embargo, las variables no pueden ser declaradas dentro de las *scenes*. Por lo que nos deja compartiendo todas las variables entre todas las escenas.

A partir de este punto, también se realizaron comprobaciones de archivos. Se aseguró que las direcciones a las que los *assets* apuntan cumplan con el tipo de archivo necesario antes de ejecutar la acción. Por ejemplo, no es posible utilizar *play* (instrucción que hace que un *asset* de sonido se ejecute) si el archivo en la ruta a la que refiere *asset* corresponde a una imagen.

Con respecto al motor, sus principales características son:

- Sistema de guardado (F5 y F9)
- Toma de decisiones
- Manejo de dialogos
- Manejo de personajes
- Manejo de fondo
- Manejo de sonido
- Input de teclado

5.3 Dificultades encontradas

Durante el desarrollo del trabajo surgieron varias dificultades. A continuación se mencionan las más relevantes.

Por un lado, una de las dificultades que surgieron fue la integración de *Raylib* al sistema de *build*, que presentó varios obstáculos. La descarga y compilación desde fuente mediante

FetchContent requirió agregar dependencias del sistema (librerías de X11, *OpenGL* y audio *ALSA*) que no estaban presentes en la imagen base. También aparecieron incompatibilidades de portabilidad entre el contenedor de compilación y el entorno de ejecución del *host* (versiones de *glibc* y de *libasan*), además de la necesidad de exponer el display gráfico para poder visualizar la ventana del motor.

Por otro lado, también representó un problema la traducción de AST a una representación intermedia (JSON) consumible por el motor y su posterior reconstrucción en tiempo de ejecución, ya que cualquier desajuste en los nombres o tipos de los campos entre lo que generaba el frontend y lo que esperaba el runtime rompía la ejecución de manera silenciosa.

A esto se sumó el diseño del modelo de ejecución del motor. Como las escenas no se modelaron como funciones, recorrer la historia implicó implementar un manejo de contextos mediante una pila, capaz de soportar *if/else* anidados, bloques de *choice* y saltos *goto* entre escenas. La correcta transición entre instrucciones —en especial al entrar a un nuevo bloque o escena— resultó ser una fuente de errores sutiles, dado que un avance mal resuelto dejaba al motor en un estado inconsistente, sin poder continuar ni finalizar la historia.

6. Futuras extensiones

Consideramos que la implementación de *loops* y de variables exclusivas para cada *scene* podrían ser muy beneficiosas para el proyecto. Además, la posibilidad de poder compilar más de un *story* con la finalidad de modularizar las escenas ayudaría a los proyectos más extensos.

Como una última adición, consideramos que la declaración de funciones que retornan valores numéricos también ayudaría a evitar repeticiones en el código y, junto con la posibilidad de usar más de un *story*, permitiría la creación de librerías escritas en el mismo lenguaje.

7. Conclusiones

Para concluir, se llevó a cabo los 3 *stages* propuestos por la cátedra. Se planeó e implementó el lenguaje planteado de la forma deseada. Principalmente se desarrolló profundamente el desarrollo en *runtime* por las necesidades del proyecto.

El proyecto permitió una vista aplicada a los temas de la materia, permitir el diseño del dominio y del lenguaje ayudó a dar una mirada más profunda a los mismos.

Finalmente, se recalca la utilidad del proyecto base otorgado por la cátedra. El uso de esta herramienta permitió un rápido comienzo con la implementación sin inconvenientes.

8. Referencias

<https://www.inklestudios.com/ink/>

El inicio del proyecto y la idea en general, vienen inspirados de la herramienta para videojuegos INK, utilizada para el manejo exclusivo de diálogos en diversos motores. A partir de este concepto inicial, se expandió a un motor propio con más funcionalidades, pero orientandonos a mantener la simpleza de nuestra referencia.

9. Bibliografía

-  (2025-09-03, v1.0.0) Análisis Léxico
-  (2024-05-08, v0.1.0) Análisis Sintáctico
-  (2024-09-25, v0.2.0) Análisis Semántico
-  (2024-05-16, v0.1.0) Generación de Código
-  (2024-05-16, v0.1.0) Runtime